

Reliable Computational Workflow Through Physics

Reliable code makes the physical dependencies and the evidence for trust visible.

Physical question. Can we construct an ideal mass-spring oscillator calculation that is readable, testable, and capable of exposing a plausible but physically wrong result?

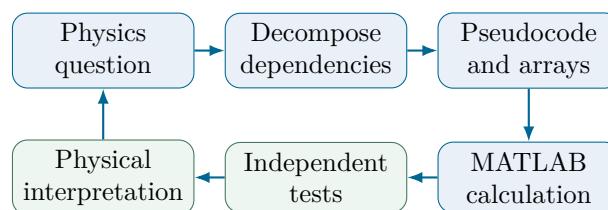
Learning outcomes

By the end of this week, you should be able to:

1. decompose a physics question into inputs, derived quantities, outputs, and checks;
2. express dependencies as structured pseudocode before implementation;
3. use vectorised MATLAB expressions while preserving units and array shapes;
4. distinguish syntax, logical, numerical, and physical defects; and
5. design several independent tests and explain what each test can detect.

1. A computational result is an argument

A program is not trustworthy merely because it runs. It makes a chain of claims: the model represents the intended physics, quantities have consistent units, the algorithm follows the model, arrays align, and the output satisfies independent expectations.



When a test fails, treat debugging as another controlled experiment. Change one suspected cause and observe whether the predicted symptom changes.

2. Worked model: an ideal mass-spring oscillator

For a horizontal oscillator with mass m , spring constant k , and displacement x from equilibrium,

$$m \frac{d^2 x}{dt^2} = -kx. \quad (1)$$

For amplitude A and phase ϕ , the analytical position and velocity are

$$x(t) = A \cos(\omega t + \phi), \quad (2)$$

$$v(t) = -A\omega \sin(\omega t + \phi), \quad (3)$$

$$\omega = \sqrt{\frac{k}{m}}, \quad T = \frac{2\pi}{\omega}. \quad (4)$$

The ideal model assumes a linear spring, constant mass, one-dimensional motion, no damping, and equilibrium at $x = 0$.

For $m = 0.50 \text{ kg}$ and $k = 8.0 \text{ N m}^{-1}$, $\omega = 4.0 \text{ rad s}^{-1}$ and $T = \pi/2 \text{ s}$. If the mass is released from rest at $x = A = 0.12 \text{ m}$, then $\phi = 0$.

Predict before computing.

- At $t = T/4$, should the velocity be positive or negative?
- What are the allowed bounds on x ?
- How should kinetic, potential, and total energy behave?

3. Decompose before writing MATLAB

Separate four kinds of information:

Layer	Oscillator example
Inputs	m, k, A, ϕ , final time, and sampling resolution
Derived quantities	$\omega = \sqrt{k/m}$ and $T = 2\pi/\omega$
Outputs	$x(t), v(t)$, kinetic energy, potential energy, and total energy
Checks	initial state, displacement bounds, one-period repeat, energy conservation, and the predicted velocity sign

Pseudocode should preserve this dependency order:

1. Define physical inputs with units.
2. Compute ω and T .
3. Create one time array.
4. Evaluate x, v, K, U , and E at the same times.
5. Apply independent checks.
6. Interpret the motion and state the model limitations.

Pseudocode is not an informal decoration. It is an explicit description of which quantities depend on which earlier decisions.

4. Vectorisation and array-shape contracts

MATLAB evaluates the analytical model for all times using arrays:

Listing 1: Vectorised oscillator calculation

```
1 mass_kg = 0.50;
2 springConstant_N_per_m = 8.0;
3 amplitude_m = 0.12;
4 omega_rad_per_s = sqrt(springConstant_N_per_m/mass_kg);
5 period_s = 2*pi/omega_rad_per_s;
6
7 t_s = linspace(0,4*period_s,1001);
8 x_m = amplitude_m.*cos(omega_rad_per_s.*t_s);
9 v_m_per_s = -amplitude_m.*omega_rad_per_s.*sin(omega_rad_per_s.*t_s);
10 K_J = 0.5*mass_kg.*v_m_per_s.^2;
11 U_J = 0.5*springConstant_N_per_m.*x_m.^2;
12 E_J = K_J + U_J;
```

The operator `.^2` squares every stored value. The arrays `t_s`, `x_m`, `v_m_per_s`, and `E_J` must have the same shape. Treat this as an interface contract:

```
1 assert(isequal(size(t_s),size(x_m),size(v_m_per_s),size(E_J)), ...
2        'The computed arrays do not share one aligned shape.')
```

Vectorisation is not automatically better physics. It is a compact way to apply the same independent calculation to aligned inputs. A scalar loop that evaluates the same formula should agree to round-off precision.

5. Layer tests that can fail independently

Testing should move from basic execution toward physical meaning.

Test layer	Example	What it can expose
Shape and finiteness	All arrays align and contain finite values	indexing and array-operation defects
Known value	$x(0) = A, v(0) = 0$	initial-condition and phase defects
Physical bound	$ x \leq A$	scaling or formula defects
Repeat case	$x(T) = x(0)$	period and frequency defects
Invariant	$K + U = \frac{1}{2}kA^2$	inconsistent position, velocity, or energy formulas
Phase prediction	$v(T/4) < 0$	a velocity-sign defect that energy alone misses

The total energy is

$$E(t) = \frac{1}{2}mv^2 + \frac{1}{2}kx^2 = \frac{1}{2}kA^2. \quad (5)$$

A missing minus sign in $v(t)$ does not change v^2 . Therefore, incorrect velocity can still pass the energy test. This is why several tests with different failure sensitivities are stronger than one impressive invariant.

Caution: a passing test is evidence, not proof. State what the test was designed to detect and identify at least one plausible error it would not detect.

6. Diagnose defects by category

Different defects require different evidence:

Category	Example	Useful first test
Syntax	unmatched bracket or invalid operator	reproduce the exact MATLAB message
Logical	<code>0:tMax:dt</code> reverses the colon arguments	inspect endpoints and number of values
Array	<code>v^2</code> uses matrix power for a vector	inspect shape; replace with <code>v.^2</code>
Numerical	exact equality is used for a sampled threshold	vary resolution or use a tolerance
Physical	$\omega = \sqrt{km}$ instead of $\sqrt{k/m}$	check dimensions and period scale
Communication	axes are reversed or unlabelled	another reader cannot interpret the output

Use a controlled debugging cycle:

1. reproduce the symptom from a clean run;
2. localise the first doubtful quantity;
3. state one hypothesis about the cause;
4. design the smallest test that distinguishes that hypothesis;
5. repair one cause and rerun all tests; and
6. explain the physical consequence of the defect.

7. Reading plausible AI-generated code

Consider this candidate:

```

1 omega_rad_per_s = sqrt(springConstant_N_per_m*mass_kg);
2 t_s = 0:4*period_s:0.01;
3 x_m = amplitude_m*cos(omega_rad_per_s*t_s);
4 energy_J = 0.5*mass_kg*v_m_per_s^2 ...
5           + 0.5*springConstant_N_per_m*x_m^2;
6 plot(x_m,t_s)

```

It contains a physical/formula defect, a logical time-array defect, array-power defects, and a communication defect. Do not ask only “Will MATLAB run this?” Ask what each line claims, which consequence follows, and which independent test would fail.

Responsible AI use. Record what Copilot was asked to do, what was accepted, modified, or rejected, why that decision was made, and which independent checks were performed. A confident explanation from AI is not validation evidence.

8. Practical transfer: projectile motion

The practical transfers the workflow to a projectile launched from height y_0 with speed v_0 and angle θ :

$$x(t) = v_0 \cos \theta t, \quad (6)$$

$$y(t) = y_0 + v_0 \sin \theta t - \frac{1}{2}gt^2. \quad (7)$$

The model assumes uniform gravity, negligible air resistance, a point-like projectile, and level ground at $y = 0$.

For $v_0 = 18.0 \text{ m s}^{-1}$, $\theta = 40^\circ$, and $y_0 = 1.5 \text{ m}$, useful references are:

Quantity	Reference value
Horizontal speed	13.7888 m s^{-1}
Initial vertical speed	11.5702 m s^{-1}
Flight time	2.48206 s
Horizontal range	34.2247 m
Peak time	1.17943 s
Peak height	8.32309 m

Strong validation evidence includes the initial position, constant horizontal velocity, the sign change of vertical velocity at the peak, the final ground condition, a reference range, aligned arrays, and finite values.

Three takeaways

1. Decomposition and pseudocode make the reasoning visible before MATLAB syntax enters.
2. Vectorisation is reliable only when units, dependencies, and array shapes remain explicit.
3. Several independent tests are needed because one plausible error may pass one strong-looking check.

Preparation for the practical. Complete the Week 02 individual Google Form first. Then bring the projectile model from physical assumptions to pseudocode, repair the supplied candidate, and defend the validation evidence. MATLAB Copilot is allowed only during group work and every material suggestion must be declared and tested.